

PATLang BNF Grammar Specification

Executive Summary

This document provides the complete Backus-Naur Form (BNF) grammar specification for the PATLang programming language. PATLang combines natural language-inspired syntax with multi-paradigm programming capabilities, including object-oriented programming, functional programming, and advanced reasoning systems.

Table of Contents

1. [Grammar Conventions](#)
2. [Core Language Grammar](#)
3. [Expression Grammar](#)
4. [Function System Grammar](#)
5. [Control Flow Grammar](#)
6. [Object-Oriented Grammar](#)
7. [Reasoning System Grammar](#)
8. [Extended Features Grammar](#)
9. [Lexical Grammar](#)
10. [Complete Grammar Reference](#)

Grammar Conventions

This BNF specification uses the following conventions:

- `<non_terminal>` - Non-terminal symbols (grammar rules)
- `"terminal"` - Terminal symbols (literal text)
- `|` - Alternative productions (OR)
- `*` - Zero or more repetitions
- `+` - One or more repetitions
- `?` - Optional (zero or one occurrence)
- `()` - Grouping
- `[]` - Optional grouping
- `{}` - Zero or more repetitions of grouped items

Core Language Grammar

Program Structure

`<program> ::= <statement_list>`

`<statement_list> ::= <statement>*`

`<statement> ::= <assignment_statement>
 | <expression_statement>
 | <function_definition>
 | <function_call_statement>
 | <control_flow_statement>
 | <reasoning_statement>
 | <print_statement>
 | <return_statement>`

`<expression_statement> ::= <expression>`

`<print_statement> ::= "print" <expression>`

`<return_statement> ::= "return" <expression>?`

Assignment Grammar

`<assignment_statement> ::= <variable_assignment>
 | <property_assignment>`

`<variable_assignment> ::= "make" <identifier> <assignment_operator> <expression>
 | "make" <identifier> <expression>
 | <identifier> <assignment_operator> <expression>`

`<property_assignment> ::= <object_reference> "." <identifier> <assignment_operator> <expression>`

`<assignment_operator> ::= "=" | "is"`

`<object_reference> ::= <identifier> ("." <identifier>)*`

Expression Grammar

Arithmetic Expressions

`<expression> ::= <logical_or_expression>`

`<logical_or_expression> ::= <logical_and_expression> ("or" <logical_and_expression>)*`

`<logical_and_expression> ::= <equality_expression> ("and" <equality_expression>)*`

`<equality_expression> ::= <relational_expression> (("==" | "!=" | "equal" | "not" "equal") <relational_expression>)*`

`<relational_expression> ::= <additive_expression> ((" "< | ">" | "<=" | ">=" | "less" | "greater" | "less=" | "greater=") <additive_expression>)*`

`<additive_expression> ::= <multiplicative_expression> (("+" | "-") <multiplicative_expression>)*`

`<multiplicative_expression> ::= <unary_expression> (("*" | "/" | "%") <unary_expression>)*`

`<unary_expression> ::= ("+" | "-" | "not" | "!") <unary_expression>
| <postfix_expression>`

`<postfix_expression> ::= <primary_expression> <postfix_operator>*`

`<postfix_operator> ::= "." <identifier>
| "." <method_call>
| "[" <expression> "]"
| "(" <argument_list>? ")"`

`<primary_expression> ::= <literal>
| <identifier>
| <function_call_expression>
| <object_creation>
| "(" <expression> ")"`

Literals

```
<literal> ::= <number_literal>  
          | <string_literal>  
          | <boolean_literal>
```

```
<number_literal> ::= <integer> | <float>
```

```
<integer> ::= <digit>+
```

```
<float> ::= <digit>+ "." <digit>+
```

```
<string_literal> ::= "\"" <string_character>* "\""
```

```
<string_character> ::= <any_character_except_quote_and_backslash>  
                    | <escape_sequence>
```

```
<escape_sequence> ::= "\\" ("n" | "t" | "r" | "\\" | "\"")
```

```
<boolean_literal> ::= "true" | "false"
```

Function System Grammar

Function Definition

```
<function_definition> ::= <simple_function_definition>  
                        | <natural_function_definition>
```

```
<simple_function_definition> ::= "function" <identifier> "(" <parameter_list>? ")" <function_body>
```

```
<natural_function_definition> ::= "make" ["a"] "function" ["called"] <identifier>  
                                ["takes" <parameter_list>]  
                                ["returns" <type_specification>]  
                                <function_body>
```

```
<parameter_list> ::= <parameter> ("," <parameter>)*
```

```
<parameter> ::= <identifier> [":" <type_specification>]
```

```
<function_body> ::= "{" <statement_list> "}"  
                | <statement>
```

```
<type_specification> ::= <basic_type>  
                       | <complex_type>  
                       | <constraint_type>
```

```
<basic_type> ::= "Number" | "String" | "Boolean" | "Object" | "Function"
```

```
<complex_type> ::= <basic_type> "[" <type_specification> "]"  
                | <basic_type> "(" <type_specification_list>? ")"
```

```
<constraint_type> ::= <type_specification> "where" <constraint_expression>
```

```
<type_specification_list> ::= <type_specification> ("," <type_specification>)*
```

Function Calls

`<function_call_statement> ::= <function_call_expression>`

`<function_call_expression> ::= <simple_function_call>
| <natural_function_call>`

`<simple_function_call> ::= <identifier> "(" <argument_list>? ")"`

`<natural_function_call> ::= "call" <identifier> ["with" <argument_list>]`

`<argument_list> ::= <expression> ("," <expression>)*`

`<method_call> ::= <identifier> "(" <argument_list>? ")"`

Control Flow Grammar

Conditional Statements

`<control_flow_statement> ::= <if_statement>
| <while_statement>`

`<if_statement> ::= "if" <expression> ["then"] <statement_block>
["else" <statement_block>]
["end"]`

`<while_statement> ::= "while" <expression> ["do"] <statement_block>
["end"]`

`<statement_block> ::= <statement>
| "{" <statement_list> "}"`

Object-Oriented Grammar

Object Creation and Manipulation

```
<object_creation> ::= "new" <identifier> ["(" <argument_list>? ")"]  
                    | "make" ["a"] <identifier> ["with" <property_list>]  
  
<property_list> ::= <property_assignment> ("," <property_assignment>)*  
  
<property_assignment_in_creation> ::= <identifier> ":" <expression>  
                                     | <identifier> "is" <expression>  
  
<object_access> ::= <object_reference> "." <identifier>  
  
<object_method_call> ::= <object_reference> "." <method_call>
```

Reasoning System Grammar

Constraints and Type Definitions

```
<reasoning_statement> ::= <constraint_definition>  
                        | <goal_definition>  
                        | <fact_assertion>  
                        | <rule_definition>  
                        | <query_statement>  
                        | <reasoning_mode_statement>  
  
<constraint_definition> ::= "constrain" <identifier> "::" <type_constraint>  
  
<type_constraint> ::= <type_specification> [<constraint_body>]  
  
<constraint_body> ::= "{" <constraint_list> "}"  
  
<constraint_list> ::= <constraint_item> ("," <constraint_item>)*  
  
<constraint_item> ::= <identifier> "::" <type_specification> ["where" <constraint_expression>]  
  
<constraint_expression> ::= <expression>
```

Goal-Oriented Programming

`<goal_definition> ::= "goal" <identifier> "(" <parameter_list>? ")" <goal_body>`

`<goal_body> ::= "{" <goal_components> "}"`

`<goal_components> ::= <goal_component>*`

`<goal_component> ::= <precondition>
 | <postcondition>
 | <strategy_specification>
 | <failure_handler>`

`<precondition> ::= "precondition:" <expression>`

`<postcondition> ::= "postcondition:" <expression>`

`<strategy_specification> ::= "strategy:" <identifier>`

`<failure_handler> ::= "failure_handler:" <identifier>`

`<goal_pursuit> ::= "pursue" <identifier> "[" <argument_list>? "]"`

Logic Programming

`<fact_assertion> ::= "fact" <predicate>`

`<rule_definition> ::= "rule" <predicate> "if" <condition_list>`

`<query_statement> ::= "query" "?-" <predicate>`

`<predicate> ::= <identifier> ["(" <term_list>? ")"]`

`<term_list> ::= <term> ("," <term>)*`

`<term> ::= <identifier>
 | <literal>
 | <compound_term>`

`<compound_term> ::= <identifier> "(" <term_list>? ")"`

`<condition_list> ::= <condition> ("and" <condition>)*`

`<condition> ::= <predicate>
 | <comparison>
 | <expression>`

`<comparison> ::= <term> <comparison_operator> <term>`

`<comparison_operator> ::= "==" | "!=" | "<" | ">" | "<=" | ">=" | "matches"`

Advanced Constructs

`<block_statement> ::= "{" <statement_list> "}"`

`<scope_block> ::= "with" <variable_declaration_list> <block_statement>`

`<variable_declaration_list> ::= <variable_declaration> ("," <variable_declaration>)*`

`<variable_declaration> ::= <identifier> [":" <type_specification>] ["=" <expression>]`

`<try_catch_statement> ::= "try" <block_statement>
 "catch" <exception_type> "(" <identifier> ")" <block_statement>`

`<exception_type> ::= <identifier>`

Lexical Grammar

Identifiers and Keywords

`<identifier> ::= <letter> (<letter> | <digit> | "_")*`

`<letter> ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m"
 | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z"
 | "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M"
 | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z"`

`<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"`

`<keyword> ::= "make" | "function" | "called" | "takes" | "returns" | "call" | "with"
 | "if" | "then" | "else" | "end" | "while" | "do" | "return" | "print"
 | "true" | "false" | "and" | "or" | "not" | "is" | "equal" | "less" | "greater"
 | "constrain" | "goal" | "fact" | "rule" | "query" | "pursue" | "reasoning" | "mode"
 | "on" | "off" | "enable" | "disable" | "precondition" | "postcondition"
 | "strategy" | "failure_handler" | "where" | "matches" | "new" | "try" | "catch"
 | "Number" | "String" | "Boolean" | "Object" | "Function"`

Operators and Punctuation

```
<operator> ::= "+" | "-" | "*" | "/" | "%" | "=" | "==" | "!=" | "<" | ">" | "<=" | ">="
            | "!" | "&&" | "||" | "::" | "?-"
```

```
<punctuation> ::= "(" | ")" | "{" | "}" | "[" | "]" | "." | "," | ";" | ":" | "?"
```

```
<delimiter> ::= <punctuation> | <operator>
```

Comments and Whitespace

```
<comment> ::= "#" <any_character_except_newline>* <newline>
```

```
<whitespace> ::= <space> | <tab> | <newline> | <carriage_return>
```

```
<space> ::= " "
```

```
<tab> ::= "\t"
```

```
<newline> ::= "\n"
```

```
<carriage_return> ::= "\r"
```

Complete Grammar Reference

Token Classification

```
<token> ::= <keyword>
          | <identifier>
          | <literal>
          | <operator>
          | <punctuation>
          | <comment>
          | <whitespace>
```

```
<literal_token> ::= <number_literal>
                  | <string_literal>
                  | <boolean_literal>
```

```
<arithmetic_operator> ::= "+" | "-" | "*" | "/" | "%"
```

```
<comparison_operator> ::= "==" | "!=" | "<" | ">" | "<=" | ">="
```

```
<logical_operator> ::= "and" | "or" | "not" | "&&" | "||" | "!"
```

```
<assignment_operator> ::= "=" | "is"
```

Natural Language Constructs

PATLang supports natural language-inspired syntax patterns:

```

<natural_assignment> ::= "make" <identifier> <expression>
                        | "make" <identifier> "is" <expression>
                        | "set" <identifier> "to" <expression>

<natural_function_call> ::= "call" <identifier>
                           | "call" <identifier> "with" <argument_list>
                           | "invoke" <identifier> "using" <argument_list>

<natural_comparison> ::= <expression> "is" "equal" "to" <expression>
                       | <expression> "is" "less" "than" <expression>
                       | <expression> "is" "greater" "than" <expression>
                       | <expression> "is" "not" "equal" "to" <expression>

<natural_condition> ::= "if" <expression> "then" <statement>
                      | "when" <expression> "do" <statement>
                      | "while" <expression> "continue" <statement>

```

Error Recovery Productions

```

<error_recovery> ::= <unexpected_token>
                    | <missing_token>
                    | <malformed_expression>

<unexpected_token> ::= <any_token_not_expected_in_context>

<missing_token> ::= <expected_token_not_found>

<malformed_expression> ::= <incomplete_expression>
                          | <invalid_operator_sequence>
                          | <mismatched_parentheses>

```

Grammar Extensions

Future Reasoning Constructs

The grammar includes provisions for future reasoning system features:

```

<advanced_reasoning> ::= <type_inference_directive>
                        | <constraint_propagation>
                        | <automated_proof>

<type_inference_directive> ::= "infer" "type" "for" <identifier>
                              | "strengthen" "type" "of" <identifier> "with" <constraint_express:

<constraint_propagation> ::= "propagate" "constraints" "from" <identifier> "to" <identifier_list>

<automated_proof> ::= "prove" <predicate> ["using" <strategy_list>]

<strategy_list> ::= <identifier> ("," <identifier>)*

```

Meta-Programming Support

```

<meta_construct> ::= <code_generation>
                    | <reflection>
                    | <introspection>

<code_generation> ::= "generate" <code_template> "for" <target_list>

<reflection> ::= "reflect" "on" <identifier>
                | "get" "metadata" "of" <identifier>

<introspection> ::= "list" "methods" "of" <identifier>
                  | "show" "type" "of" <identifier>
                  | "describe" <identifier>

```

Production Rules Summary

This grammar specification defines approximately:

- **150+ production rules** covering all language constructs
- **80+ terminal symbols** including keywords, operators, and punctuation
- **Natural language syntax** for improved readability
- **Multi-paradigm support** for OOP, functional, and reasoning programming
- **Error recovery mechanisms** for robust parsing
- **Extension points** for future language features

Implementation Notes

Parser Implementation Guidance

1. **Left-to-Right Parsing:** The grammar is designed for LL(k) parsing with minimal lookahead
2. **Operator Precedence:** Arithmetic and logical operators follow standard precedence rules
3. **Ambiguity Resolution:** Natural language constructs may require context-sensitive parsing
4. **Error Recovery:** Productions include error recovery patterns for robust parsing
5. **Extensibility:** Grammar structure supports incremental feature addition

Semantic Constraints

The grammar defines syntactic structure. Additional semantic constraints include:

- **Type Compatibility:** Assignment operations must respect type constraints
- **Scope Rules:** Variable references must be in scope
- **Function Signatures:** Function calls must match defined signatures
- **Reasoning Consistency:** Logic programming constructs must be logically consistent

Cross-References

- **Lexer Implementation:** `src/lexer.rb`
- **Parser Implementation:** `src/parser.rb`
- **Token Definitions:** `src/token.rb`
- **AST Node Definitions:** `src/ast_nodes.rb`
- **Language Examples:** `examples/unified-reasoning-examples.md`

Conclusion

This BNF grammar specification provides a complete formal definition of the PATLang language syntax. The grammar supports PATLang's unique combination of natural language inspiration, multi-paradigm programming, and advanced reasoning capabilities while maintaining the precision needed for reliable parser implementation.

The grammar is designed to be both human-readable and machine-parseable, reflecting PATLang's philosophy of making programming more accessible while retaining the power needed for complex reasoning and computational tasks.